

```

In [1]: library(Seurat)

library(reticulate)
library(anndata)
library(ggplot2)
library(ggrepel)
library(ggpubr)
library(pheatmap)
library(dplyr)
library(tidyr)
library(RColorBrewer)
library(clustree)
library(repr)
library(scico)
options(repr.plot.width=10, repr.plot.height=8)

library(UpSetR)
library(grid)

library(PRRoc)
library(Matrix)

getwd()

dataset_id <- "simulated_mm_RA"
genome_id <- "mm10"
samples <- c("all", "old", "young")
colorSamples <- c("old"="#A58065",
                  "young"="#7FCFF2",
                  "all"="#92A8AC")
dir.create(paste0("figures_", dataset_id))
for (sample in samples) {
  dir.create(paste0("figures_", dataset_id, "_", sample))
}

colorTools <- c(
  "STARsolo_TE" = "#FFBC81",
  "STARsolo_TE_EM" = "#F3E088",
  "SoloTE_unique" = "#A4DD9B",
  "SoloTE_thr2" = "#6FC69D",
  "SoloTE_thr1" = "#4BB2BB",
  "SoloTE_thr0" = "#3989BF",
  "Stellarscope" = "#4F5D93",
  "simulated" = "grey70"
)

thrMinCells <- 500 * 0.05

```

Loading required package: SeuratObject

Loading required package: sp

'SeuratObject' was built under R 4.3.3 but the current version is 4.4.1; it is recommended that you reinstall 'SeuratObject' as the ABI for R may have changed

'SeuratObject' was built with package 'Matrix' 1.6.4 but the current version is 1.7.5; it is recommended that you reinstall 'SeuratObject' as the ABI for 'Matrix' may have changed

Attaching package: 'SeuratObject'

The following objects are masked from 'package:base':

intersect, t

Attaching package: 'anndata'

The following object is masked from 'package:SeuratObject':

Layers

Warning message:

"package 'dplyr' was built under R version 4.4.3"

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Warning message:

"package 'tidyr' was built under R version 4.4.3"

Warning message:

"package 'RColorBrewer' was built under R version 4.4.3"

Loading required package: ggraph

Attaching package: 'ggraph'

The following object is masked from 'package:sp':

geometry

Loading required package: rlang

Warning message:

"package 'rlang' was built under R version 4.4.3"

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

'/mnt/TEdata/TEbenchmarking/Rscripts'

Warning message in dir.create(paste0("figures_", dataset_id)):

"'figures_simulated_mm_RA' already exists"

Warning message in dir.create(paste0("figures_", dataset_id, "_", sample)):

"'figures_simulated_mm_RA_all' already exists"

Warning message in dir.create(paste0("figures_", dataset_id, "_", sample)):

"'figures_simulated_mm_RA_old' already exists"

Warning message in dir.create(paste0("figures_", dataset_id, "_", sample)):

"'figures_simulated_mm_RA_young' already exists"

```
In [2]: theme_paper <- function(base_size = 17, base_family = "") {
  theme_minimal(base_size = base_size, base_family = base_family) +
  theme(
    plot.title = element_text(face = "bold", size = base_size + 2, hjust = 0.5),
    axis.title = element_text(size = base_size),
    axis.text = element_text(size = base_size * 0.9),
    legend.title = element_text(size = base_size),
    legend.text = element_text(size = base_size * 0.9),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    plot.margin = margin(10, 10, 10, 10)
  )
}
```

```
In [2]: load("workspaces/00_evaluation_objectCreation.Rdata")
```

```
In [4]: # compute TE length
conversion_table$TElength <- conversion_table$end - conversion_table$start
```

```
In [6]: # load truncation annotation (uploaded to Zenodo)
annotation_truncation <- read.table("annotation/annotated_tes_mm10.tsv", header = TRUE)

table(annotation_truncation$status)
table(annotation_truncation$annotation)

# Full_Length – the insertion covers ≥90% of the repeat consensus sequence
# 5prime_Truncated – >10% of the consensus is missing from the 5' end only
# 3prime_Truncated – >10% of the consensus is missing from the 3' end only
# Internal_Fragment – >10% missing from both ends; only the middle of the element is present
# Truncated_Other – coverage <90% but neither end gap exceeds 10%; typically a slightly degraded or ambiguously

colnames(annotation_truncation) <- gsub(colnames(annotation_truncation), pattern="genoName", replacement="chr")
colnames(annotation_truncation) <- gsub(colnames(annotation_truncation), pattern="genoStart", replacement="start")
colnames(annotation_truncation) <- gsub(colnames(annotation_truncation), pattern="genoEnd", replacement="end")
annotation_truncation$start <- annotation_truncation$start + 1
colnames(annotation_truncation)

Full_Length    Truncated
2448693        2885046
3prime_Truncated 5prime_Truncated    Full_Length Internal_Fragment
701149          1282138    2448693          826993
Truncated_Other
74766

'bin' · 'swScore' · 'milliDiv' · 'milliDel' · 'milliIns' · 'chr' · 'start' · 'end' · 'genoLeft' · 'strand' · 'repName' · 'repClass' ·
'repFamily' · 'repStart' · 'repEnd' · 'repLeft' · 'id' · 'cons_len' · 'aln_len' · 'coverage' · 'status' · 'annotation'
```

```
In [7]: # add truncation to the TE annotation table
conversion_table <- merge(conversion_table,
                          annotation_truncation[,c("chr", "start", "end", "strand", "coverage", "status", "annotation")],
                          all.x=T)
dim(conversion_table)

3727460 · 20
```

```
In [8]: annotationYoungSim <- conversion_table[conversion_table$stellarscopeID %in% rownames(splatter_objs[["young"]]),]
annotationOldSim <- conversion_table[conversion_table$stellarscopeID %in% rownames(splatter_objs[["old"]]),]
```

Locus characteristics

```
In [10]: familyOrder <- c("L1", 'ERVL', 'ERVK', 'ERV1', 'ERVL-MaLR', 'B2', 'Alu')

annotationYoungSim$family <- factor(annotationYoungSim$family, levels = rev(familyOrder))

# simulated loci

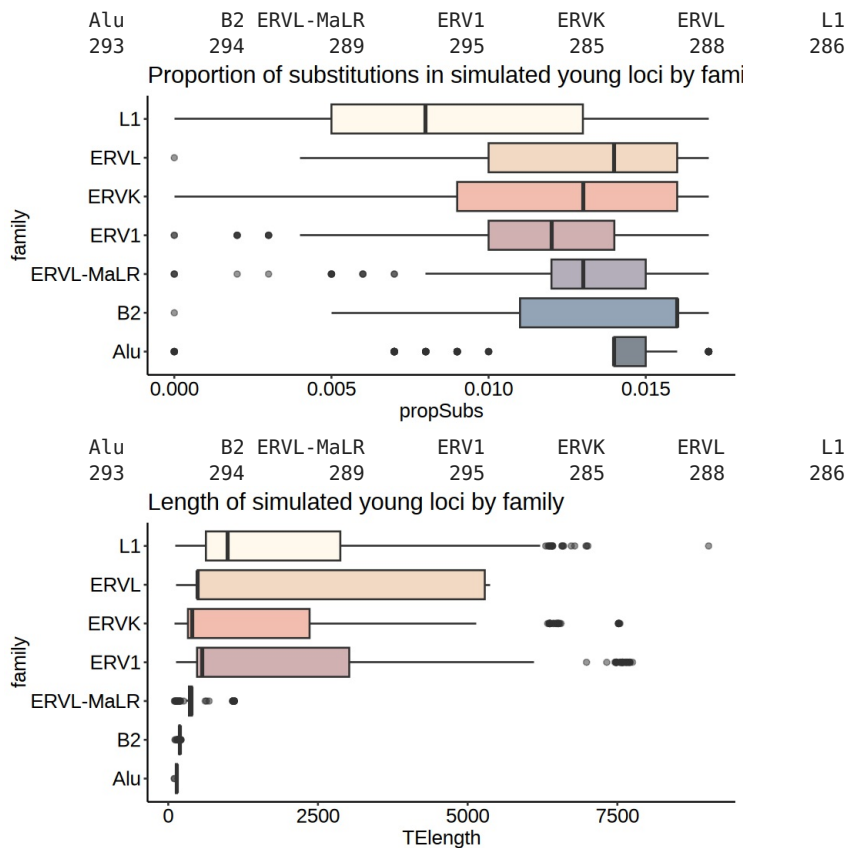
options(repr.plot.width=8, repr.plot.height=4)
ggplot(annotationYoungSim, aes(x=propSubs, y=family, fill=family)) +
  geom_boxplot(alpha=0.5) +
  scale_fill_scico_d(palette="lipari") +
  guides(fill = "none") +
  theme_pubr() +
  theme(text=element_text(size=15)) +
  ggtitle("Proportion of substitutions in simulated young loci by family")
table(annotationYoungSim$family)
ggsave("figures_simulated_mm_RA_young/distributionOfProbSub.pdf", device="pdf", width=8, height=4)

options(repr.plot.width=8, repr.plot.height=4)
ggplot(annotationYoungSim, aes(x=TElength, y=family, fill=family)) +
  geom_boxplot(alpha=0.5) +
```

```

scale_fill_scico_d(palette="lipari") +
guides(fill = "none") +
theme_pubr() +
theme(text=element_text(size=15)) +
ggtitle("Length of simulated young loci by family")
table(annotationYoungSim$family)
ggsave("figures_simulated_mm_RA_young/distributionOfTElengths.pdf", device="pdf", width=8, height=6)

```



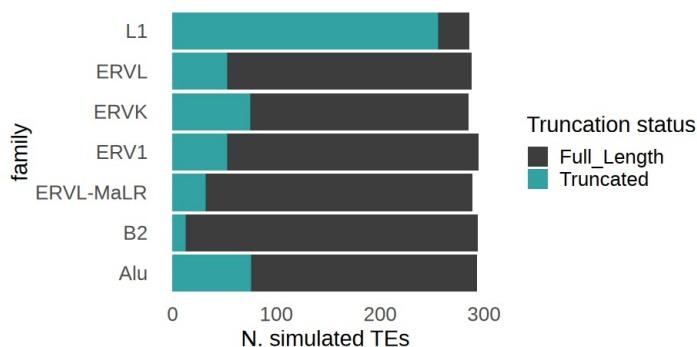
```

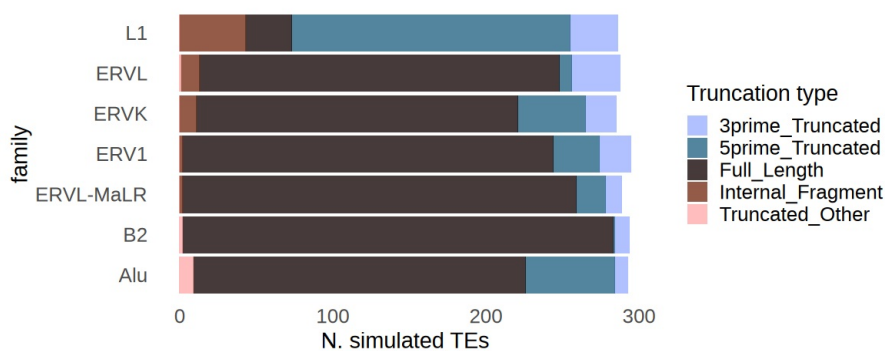
In [11]: options(repr.plot.width=8, repr.plot.height=4)
ggplot(as.data.frame(table(annotationYoungSim$status, annotationYoungSim$family)), aes(x=Freq, y=Var2, fill=Var1)) +
  ylab("family") +
  xlab("N. simulated TEs") +
  labs(fill = "Truncation status") +
  scale_fill_manual(values=c("grey5", "darkcyan")) +

  geom_col(alpha = 0.8) +
  theme_paper()
ggsave("figures_simulated_mm_RA_young/distributionOfTruncationStatus_allYoung.pdf", device="pdf", width=8, height=4)

options(repr.plot.width=10, repr.plot.height=4)
ggplot(as.data.frame(table(annotationYoungSim$annotation, annotationYoungSim$family)), aes(x=Freq, y=Var2, fill=Var1)) +
  ylab("family") +
  labs(fill = "Truncation type") +
  xlab("N. simulated TEs") +
  geom_col(alpha = 0.8) +
  scale_fill_scico_d(palette="berlin") +
  theme_paper()
ggsave("figures_simulated_mm_RA_young/distributionOfTruncationType_allYoung.pdf", device="pdf", width=10, height=4)

```





Evaluation

Upset plots of detected TEs and TPs

Upsets of detected TEs

```
In [8]: options(repr.plot.width=12, repr.plot.height=6)

TPlistSample <- list()
FPlistSample <- list()
FNlistSample <- list()

for (sample in samples){
  print(sample)

  detectedList <- list()

  TPlistSample[[sample]] <- list()
  FPlistSample[[sample]] <- list()
  FNlistSample[[sample]] <- list()

  for (tool in c(names(objList), "simulated")){
    print(tool)
    if (tool == "simulated") {
      obj <- splatter_objs[[sample]]
    }else {
      obj <- objList[[tool]][[sample]]
    }

    detectedList[[sub("^(.*)_.*$", "\\1", obj@project.name)]] <- Features(obj)
    TPlistSample[[sample]][[sub("^(.*)_.*$", "\\1", obj@project.name)]] <- intersect(Features(splatter_objs
    FPlistSample[[sample]][[sub("^(.*)_.*$", "\\1", obj@project.name)]] <- setdiff(Features(obj), Features(
    FNlistSample[[sample]][[sub("^(.*)_.*$", "\\1", obj@project.name)]] <- setdiff(Features(splatter_objs[[

  }

  #pdf(file=paste0("figures_",dataset_id,"_",sample,"/upset_detected.pdf"), width = 13, height = 7.5)
  show(UpSetR::upset(fromList(detectedList), nintersects = 30,
    sets=names(colorTools), keep.order = T, sets.bar.color=colorTools,
    text.scale = c(2, 2, 2, 1.65, 2.5, 1.5),
    order.by=c("freq"),
    point.size=3, mb.ratio = c(0.5, 0.5),
    set_size.show=F, set_size.scale_max= max(lengths(detectedList))+1000, #show.numbers = F,
    nsets=length(objList)+1,
    sets.x.label="N. detected loci",
    mainbar.y.label="Intersection size")

  )
  grid.text(paste0("Detected TEs - ", sample ),x = 0.65, y=0.95, gp=gpar(fontsize=18))
  #dev.off()

  #pdf(file=paste0("figures_",dataset_id,"_",sample,"/upset_detected_TP.pdf"), width = 11, height = 7.5)
  show(UpSetR::upset(fromList(TPlistSample[[sample]]), nintersects = 20,
    sets=names(colorTools), keep.order = T, sets.bar.color=colorTools,
    text.scale = c(2, 2, 2, 1.65, 2.2, 1.5),
    order.by=c("freq"),
```

```

    point.size=3, mb.ratio = c(0.5, 0.5),
    set_size.show=F, set_size.scale_max= max(lengths(TPlistSample[[sample]]))+1000, #show.numbers = F,
    nsets=length(c(objList, splatter_obj)),
    sets.x.label="N. correctly detected loci",
    mainbar.y.label="Intersection size")
  )
  grid.text(paste0("TP TEs - ", sample ),x = 0.65, y=0.95, gp=gpar(fontsize=18))

  #dev.off()
}

```

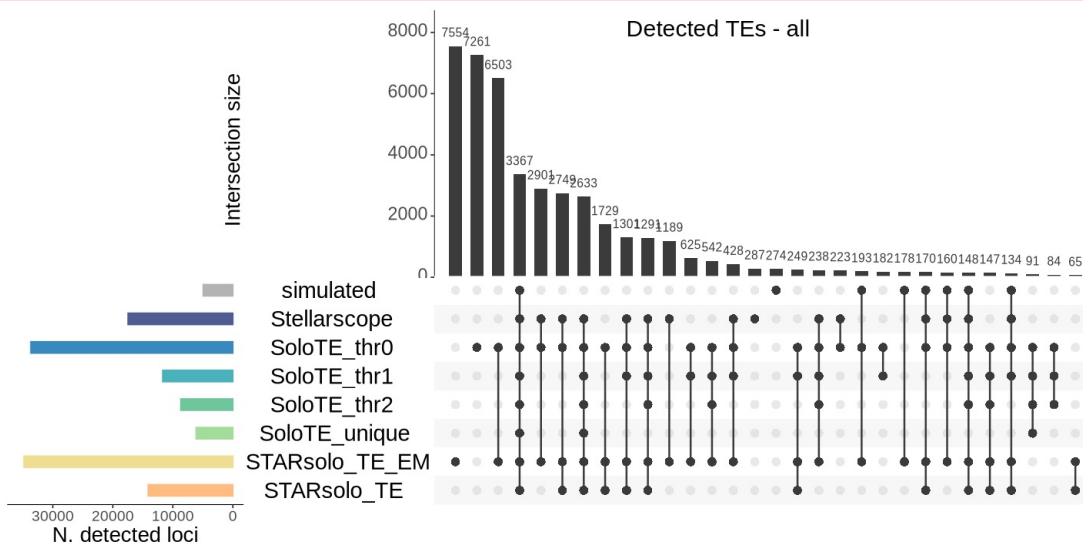
```

[1] "all"
[1] "STARsolo_TE"
[1] "STARsolo_TE_EM"
[1] "SoloTE_unique"
[1] "SoloTE_thr2"
[1] "SoloTE_thr1"
[1] "SoloTE_thr0"
[1] "Stellarscope"
[1] "simulated"

```

Warning message:
 "`aes\_string()` was deprecated in ggplot2 3.0.0.
 Please use tidy evaluation idioms with `aes()``.
 See also `vignette("ggplot2-in-packages")` for more information.
 The deprecated feature was likely used in the [UpSetR](#) package.
 Please report the issue to the authors."

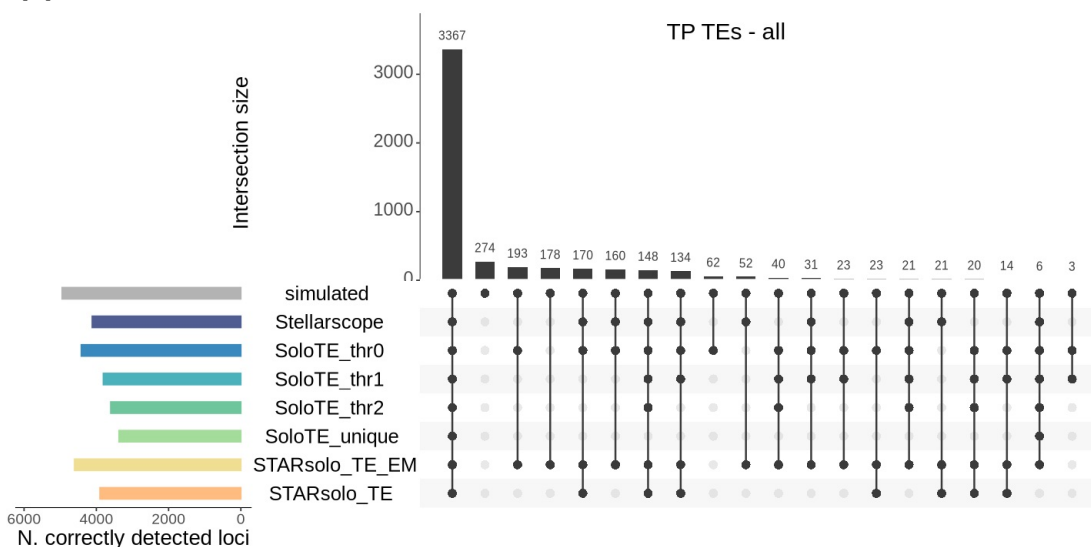
Warning message:
 "Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 Please use `linewidth` instead.
 The deprecated feature was likely used in the [UpSetR](#) package.
 Please report the issue to the authors."

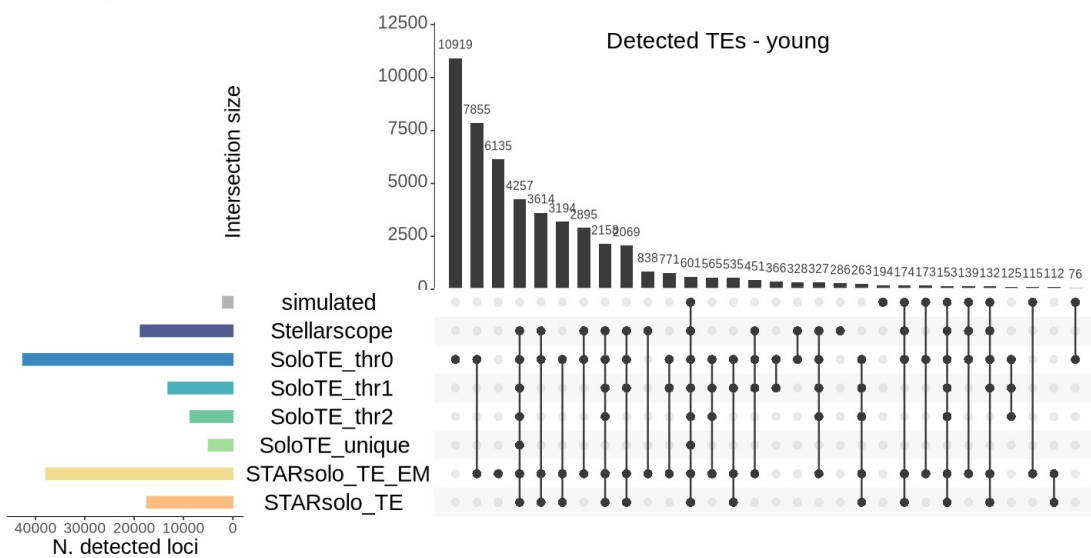
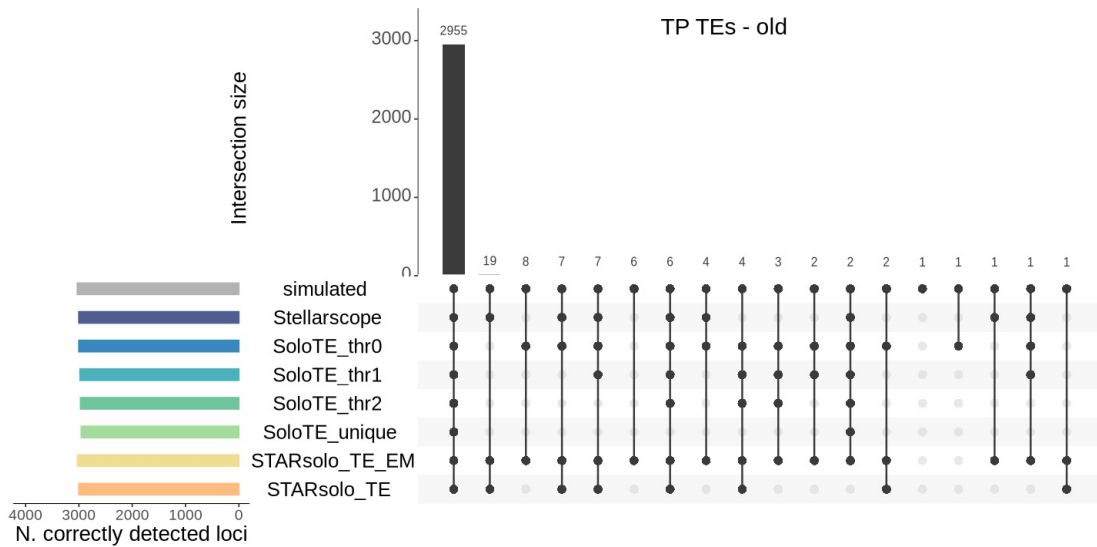
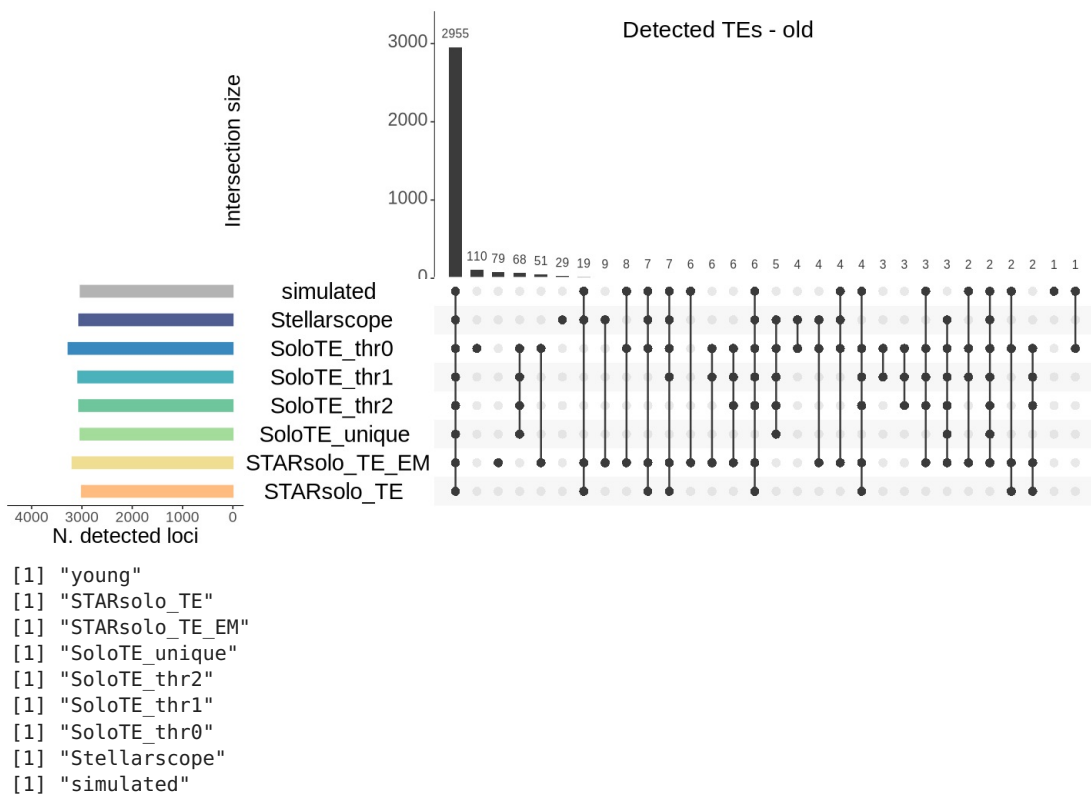


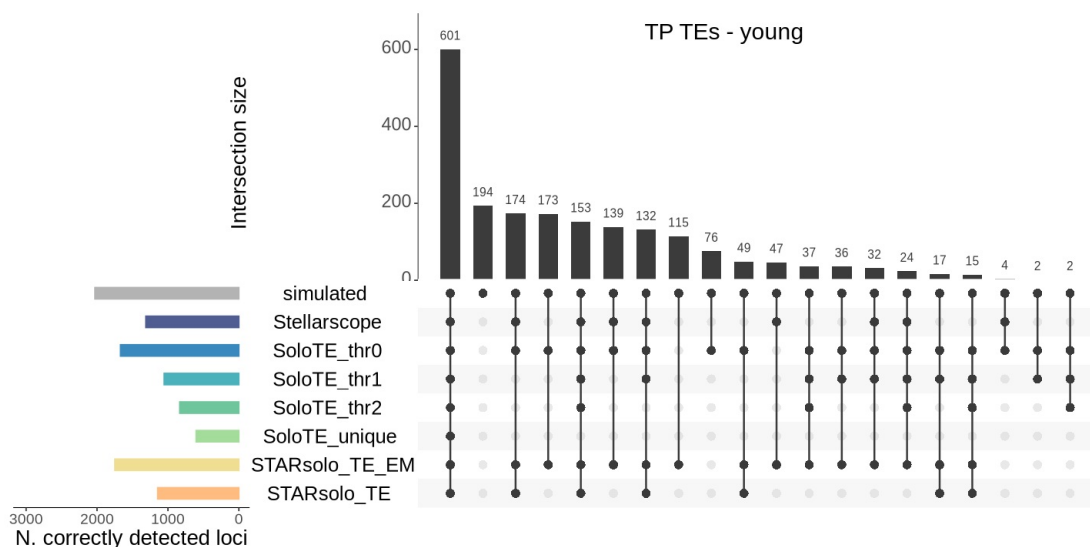
```

[1] "old"
[1] "STARsolo_TE"
[1] "STARsolo_TE_EM"
[1] "SoloTE_unique"
[1] "SoloTE_thr2"
[1] "SoloTE_thr1"
[1] "SoloTE_thr0"
[1] "Stellarscope"
[1] "simulated"

```







Common biases

```
In [9]: sample <- "young"

# get common false positives
FPlistSample[[sample]][["simulated"]] <- NULL
lengths(FPlistSample[[sample]])
commonFPs <- Reduce(intersect, FPlistSample[[sample]])
length(commonFPs)

# get common false negatives
FNlistSample[[sample]][["simulated"]] <- NULL
lengths(FNlistSample[[sample]])
commonFNs <- Reduce(intersect, FNlistSample[[sample]])
length(commonFNs)
```

STARsolo_TE: 16265 **STARsolo_TE_EM:** 36106 **SoloTE_unique:** 4332 **SoloTE_thr2:** 7803 **SoloTE_thr1:** 12062
SoloTE_thr0: 40867 **Stellarscope:** 17416

4257

STARsolo_TE: 885 **STARsolo_TE_EM:** 281 **SoloTE_unique:** 1427 **SoloTE_thr2:** 1196 **SoloTE_thr1:** 976 **SoloTE_thr0:**
361 **Stellarscope:** 718

194

```
In [10]: fp <- conversion_table %>% filter(stellarscopeID %in% commonFPs)
tp <- conversion_table %>% filter(stellarscopeID %in% rownames(splatter_objs[["young"]]))
fn <- conversion_table %>% filter(stellarscopeID %in% commonFNs)

fp$label <- "common FPs"
tp$label <- "simulated loci"
fn$label <- "common FNs"
```

```
In [11]: options(repr.plot.width=5, repr.plot.height=6)

library(scico)
library(ggbeeswarm)

df <- rbind(fp, tp, fn)
df$label <- factor(df$label, levels=c("simulated loci", "common FNs", "common FPs"))

ggplot(df, aes(label, TElength)) +
  geom_boxplot() +
  stat_summary(fun.data = function(x) data.frame(y = max(x), label = length(x)),
    geom = "text", vjust = -0.5, size = 5) +
  theme_light() +
  xlab("") +
  theme(text=element_text(size=18),
    axis.text.x=element_text(angle=30, hjust=1))
ggsave(paste0("figures_", dataset_id, "_", sample, "/TElength_commonFP_commonFN.pdf"), width = 4, height = 4)
```



```

ggplot(df, aes(x=label, y=TElength)) +
  geom_violin() +
  stat_summary(
    fun.data = function(x) data.frame(y = max(x), label = sprintf("n: %d", length(x))),
    geom = "text",
    vjust = -0.5,
    size = 5
  ) +
  stat_summary(
    fun = median,
    geom = "crossbar",
    width = 0.2,
    color = "grey30",
    linewidth = 0.5
  ) +
  stat_summary(
    fun = median,
    geom = "text",
    aes(label = round(after_stat(y), 2)),
    hjust = -0.4, # Positive values shift text right; negative values shift it left
    vjust = 0.5, # Centers the text vertically with the crossbar line
    size = 5,
    color = "grey30"
  ) +
  theme_pubr() +
  xlab("") +
  theme(
    text = element_text(size = 18),
    axis.text.x = element_text(angle = 30, hjust = 1)
  )
ggsave(paste0("figures_", dataset_id, "_", sample, "/TElength_commonFP_commonFN_violin.pdf"), width = 5, height = 6)

ggplot(df, aes(x=label, y=propSubs)) +
  geom_boxplot() +
  stat_summary(fun.data = function(x) data.frame(y = max(x), label = length(x)),
    geom = "text", vjust = -0.5, size = 5) +
  theme_light() +
  xlab("") +
  theme(text=element_text(size=18),
    axis.text.x=element_text(angle=30, hjust=1))
ggsave(paste0("figures_", dataset_id, "_", sample, "/propSubs_commonFP_commonFN.pdf"), width = 4, height = 4)

ggplot(df, aes(label, propSubs)) +
  geom_violin() +
  stat_summary(
    fun.data = function(x) data.frame(y = max(x), label = sprintf("n: %d", length(x))),
    geom = "text",
    vjust = -0.5,
    size = 5
  ) +
  stat_summary(
    fun = median,
    geom = "crossbar",
    width = 0.2,
    color = "grey30",
    linewidth = 0.5
  ) +
  stat_summary(
    fun = median,
    geom = "text",
    aes(label = round(after_stat(y), 2)),
    hjust = -0.4, # Positive values shift text right; negative values shift it left
    vjust = 0.5, # Centers the text vertically with the crossbar line
    size = 5,
    color = "grey30"
  ) +
  theme_pubr() +
  xlab("") +
  theme(
    text = element_text(size = 18),
    axis.text.x = element_text(angle = 30, hjust = 1)
  )
ggsave(paste0("figures_", dataset_id, "_", sample, "/propSubs_commonFP_commonFN_violin.pdf"), width = 4, height = 4)

ggplot(df, aes(label, mya)) +
  geom_boxplot() +
  stat_summary(fun.data = function(x) data.frame(y = max(x), label = length(x)),
    geom = "text", vjust = -0.5, size = 5) +
  theme_light() +
  theme(text=element_text(size=18),
    axis.text.x=element_text(angle=30, hjust=1))
ggsave(paste0("figures_", dataset_id, "_", sample, "/mya_commonFP_commonFN.pdf"), width = 4, height = 4)

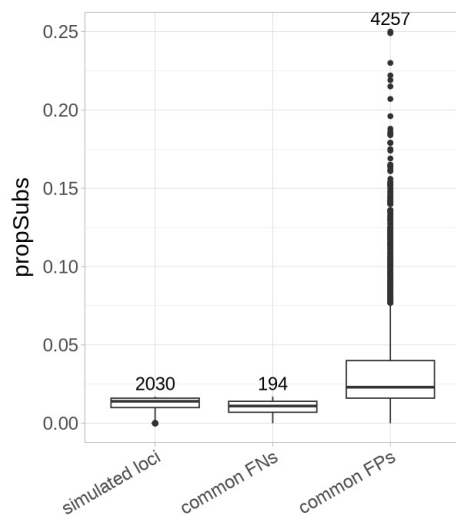
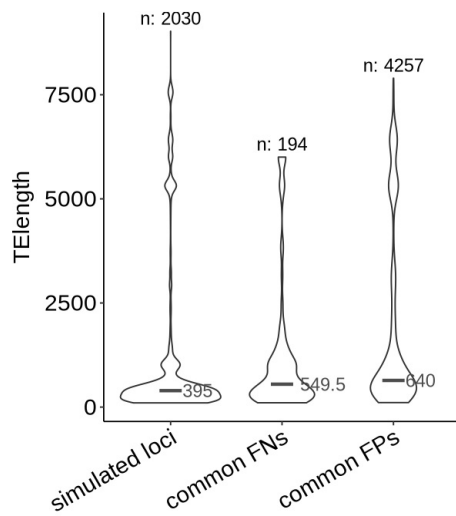
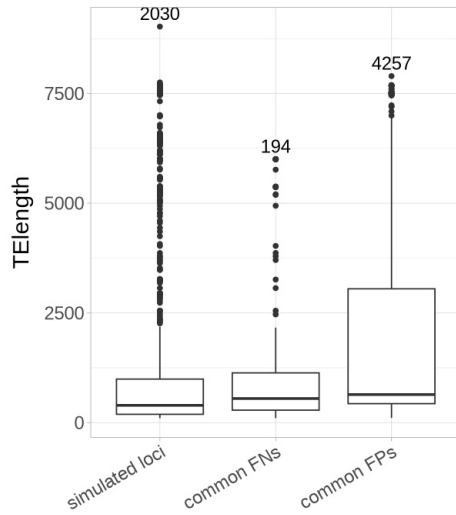
```

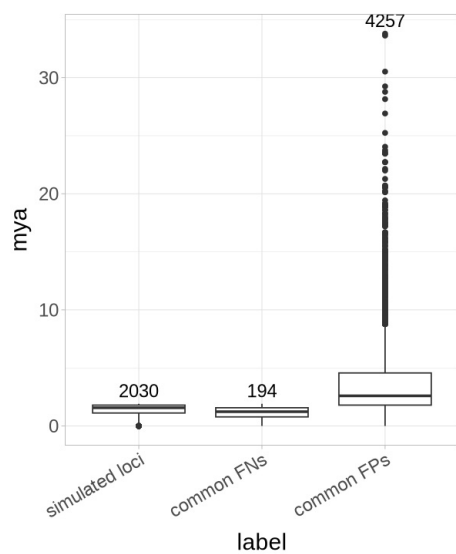
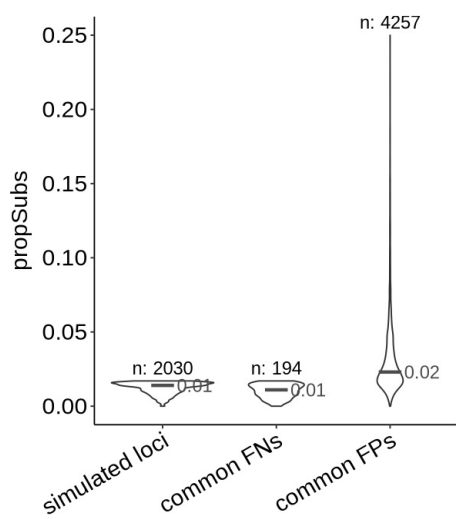
```

ggplot(df, aes(family, fill = label)) +
  geom_bar(position = "fill") +
  theme_pubclean() +
  theme(text=element_text(size=18),
        axis.text.x = element_text(angle=30, hjust=1))
ggsave(paste0("figures_",dataset_id,"_",sample,"/familyComp_commonFP_commonFN.pdf"), width = 4, height = 4)

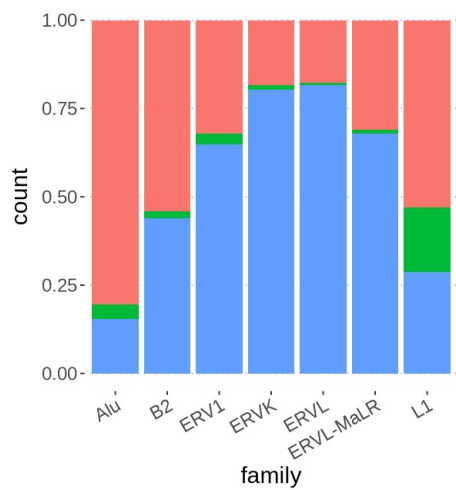
options(repr.plot.width=6, repr.plot.height=7)
ggplot(df, aes(label, fill = family)) +
  geom_bar(position = "fill") +
  scale_fill_scico_d(palette="lipari") +
  xlab("") +
  theme_pubr() +
  theme(text=element_text(size=18),
        axis.text.x = element_text(angle=30, hjust=1))
ggsave(paste0("figures_",dataset_id,"_",sample,"/familyEnrichment_commonFP_commonFN.pdf"), width = 6, height = 6)

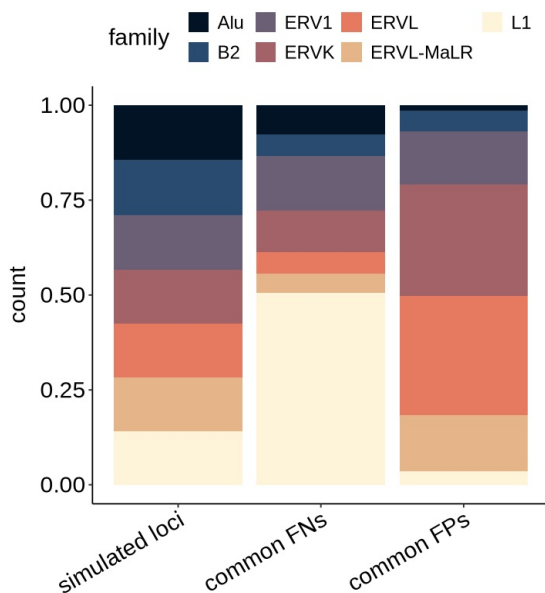
```





label ■ simulated loci ■ common FNs ■ common





Common FPs

```
In [12]: # common FPs
df <- rbind(fp, tp)

df %>%
  group_by(label) %>%
  summarise(
    n = n(),
    median_length = median(TLength),
    median_propSubs = median(propSubs),
    mean_age = mean(mya)
  )

wilcox.test(TLength ~ label, data = df)
wilcox.test(propSubs ~ label, data = df)
wilcox.test(mya ~ label, data = df)

table(df$family, df$label)
chisq.test(table(df$family, df$label))
```

A tibble: 2 × 5

	label	n	median_length	median_propSubs	mean_age
	<chr>	<int>	<dbl>	<dbl>	<dbl>
	common FPs	4257	640	0.023	3.781206
	simulated loci	2030	395	0.014	1.374822

Wilcoxon rank sum test with continuity correction

data: TLength by label
W = 5845263, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
Wilcoxon rank sum test with continuity correction

data: propSubs by label
W = 7453312, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0

Wilcoxon rank sum test with continuity correction

data: mya by label
W = 7453312, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0

	common FPs	simulated loci
Alu	56	293
B2	238	294
ERV1	594	295
ERVK	1252	285
ERVL	1332	288
ERVL-MaLR	630	289
L1	155	286

Pearson's Chi-squared test

data: table(df\$family, df\$label)
X-squared = 1057.9, df = 6, p-value < 2.2e-16

```
In [13]: library(rstatix)
          wilcox_effsize(df, TElength ~ label)
          wilcox_effsize(df, propSubs ~ label)
          wilcox_effsize(df, mya ~ label)
```

Attaching package: 'rstatix'

The following object is masked from 'package:stats':

filter

A rstatix_test: 1 × 7

	.y.	group1	group2	effsize	n1	n2	magnitude
	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<ord>
Effect size (r)	TElength	common FPs	simulated loci	0.2857151	4257	2030	small

A rstatix_test: 1 × 7

	.y.	group1	group2	effsize	n1	n2	magnitude
	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<ord>
Effect size (r)	propSubs	common FPs	simulated loci	0.5877846	4257	2030	large

A rstatix_test: 1 × 7

	.y.	group1	group2	effsize	n1	n2	magnitude
	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<ord>
Effect size (r)	mya	common FPs	simulated loci	0.5877846	4257	2030	large

Common FNs

```
In [14]: # common FNs
          df <- rbind(fn, tp)

          df %>%
            group_by(label) %>%
            summarise(
              n = n(),
              median_length = median(TElength),
              median_propSubs = median(propSubs),
              mean_age = mean(mya)
            )

          wilcox.test(TElength ~ label, data = df)
          wilcox.test(propSubs ~ label, data = df)
          wilcox.test(mya ~ label, data = df)

          table(df$family, df$label)
          chisq.test(table(df$family, df$label))
```

A tibble: 2 × 5

label	n	median_length	median_propSubs	mean_age
<chr>	<int>	<dbl>	<dbl>	<dbl>
common FNs	194	549.5	0.011	1.148732
simulated loci	2030	395.0	0.014	1.374822

Wilcoxon rank sum test with continuity correction

data: TLength by label

W = 221996, p-value = 0.003327

alternative hypothesis: true location shift is not equal to 0

Wilcoxon rank sum test with continuity correction

data: propSubs by label

W = 148078, p-value = 8.844e-09

alternative hypothesis: true location shift is not equal to 0

Wilcoxon rank sum test with continuity correction

data: mya by label

W = 148078, p-value = 8.844e-09

alternative hypothesis: true location shift is not equal to 0

	common FNs	simulated loci
Alu	15	293
B2	11	294
ERV1	28	295
ERVK	21	285
ERVL	11	288
ERVL-MaLR	10	289
L1	98	286

Pearson's Chi-squared test

data: table(df\$family, df\$label)

X-squared = 173.6, df = 6, p-value < 2.2e-16

```
In [15]: library(rstatix)
wilcox_effsize(df, TLength ~ label)
wilcox_effsize(df, propSubs ~ label)
wilcox_effsize(df, mya ~ label)
```

A rstatix_test: 1 × 7

	.y.	group1	group2	effsize	n1	n2	magnitude
	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<ord>
Effect size (r)	TLength	common FNs	simulated loci	0.06225423	194	2030	small

A rstatix_test: 1 × 7

	.y.	group1	group2	effsize	n1	n2	magnitude
	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<ord>
Effect size (r)	propSubs	common FNs	simulated loci	0.1219609	194	2030	small

A rstatix_test: 1 × 7

	.y.	group1	group2	effsize	n1	n2	magnitude
	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<ord>
Effect size (r)	mya	common FNs	simulated loci	0.1219609	194	2030	small

Common FP and FN chi square test

```
In [16]: df <- rbind(fp, tp, fn)

tab <- table(df$family, df$label)
tab

chisq <- chisq.test(tab)

chisq

chisq$expected

chisq$stdres

library(rstatix)
cramer_v(tab)
```

	common FNs	common FPs	simulated loci
Alu	15	56	293
B2	11	238	294
ERV1	28	594	295
ERVK	21	1252	285
ERVL	11	1332	288
ERVL-MaLR	10	630	289
L1	98	155	286

Pearson's Chi-squared test

data: tab

X-squared = 1514.8, df = 12, p-value < 2.2e-16

A matrix: 7 × 3 of type dbl

	common FNs	common FPs	simulated loci
Alu	10.89585	239.0909	114.0133
B2	16.25397	356.6658	170.0802
ERV1	27.44916	602.3251	287.2257
ERVK	46.63663	1023.3615	488.0019
ERVL	48.82179	1071.3111	510.8672
ERVL-MaLR	27.80836	610.2072	290.9844
L1	16.13424	354.0384	168.8273

	common FNs	common FPs	simulated loci
Alu	1.2994028	-20.8060960	20.8203065
B2	-1.3823190	-11.2059672	11.9786200
ERV1	0.1152095	-0.6249644	0.5974028
ERVK	-4.3732341	13.9989189	-12.7229427
ERVL	-6.3530845	15.7169607	-13.7541712
ERVL-MaLR	-3.7045192	1.4778080	-0.1516653
L1	21.6113569	-18.8590592	11.3645382

0.341854396406496

Precision / Recall

Thr 0 counts

```
In [14]: precisionTool <- list()
recallTool <- list()
# TPtool <- list()
# FPtool <- list()
# FNtool <- list()

for (sample in samples){
  layer <- "counts"

  matSplatter <- GetAssayData(splatter_objs[[sample]], layer=layer)

  precisionTool[[sample]] <- list()
  recallTool[[sample]] <- list()
  # FPtool[[sample]] <- list()
  # FNtool[[sample]] <- list()

  for(tool in names(objList)){

    matTool <- GetAssayData(objList[[tool]][[sample]], layer=layer)

    precisionTool[[sample]][[tool]] <- NULL
    recallTool[[sample]][[tool]] <- NULL

    for(cell in Cells(splatter_objs[[sample]])){

      exprSplatter <- names(matSplatter[,cell])[matSplatter[,cell] > 0]
      # TEs expressed in the simulated matrix

      exprTool <- names(matTool[,cell])[matTool[,cell] > 0]
      # TEs detected by the tool

      TP <- intersect(exprSplatter, exprTool)
      #TPtool[[sample]][[tool]] <- TP
      nTP <- length(TP)

      FP <- setdiff(exprTool, exprSplatter)
      #FPtool[[sample]][[tool]] <- FP
    }
  }
}
```

```

nFP <- length(FP)

# add FP genes to the FP count
# if(tool %in% names(objGeneList)){
#     FP_genes <- names(matTool_genes[,cell])[matTool_genes[,cell] > 0]
#     nFP_genes <- length(FP_genes)
#     nFP <- nFP + nFP_genes
# }

FN <- setdiff(exprSplatter, exprTool)
# FNtool[[sample]][[tool]] <- FN
nFN <- length(FN)

precision <- nTP / (nTP + nFP)
precisionTool[[sample]][[tool]] <- c(precisionTool[[sample]][[tool]], precision)

recall <- nTP / (nTP + nFN)
recallTool[[sample]][[tool]] <- c(recallTool[[sample]][[tool]], recall)
}
}
}

```

In [21]: `options(repr.plot.width=5, repr.plot.height=3)`

```

for (sample in samples){
  print(sample)

  precisionDf <- stack(precisionTool[[sample]])
  colnames(precisionDf) <- c("precision", "tool")

  recallDf <- stack(recallTool[[sample]])
  colnames(recallDf) <- c("recall", "tool")

  df <- merge(precisionDf, recallDf, by='tool')

  df$F1score <- (2 * df$precision * df$recall) / (df$precision + df$recall)

  df$tool <- factor(df$tool, levels=rev(names(colorTools)))

  # plot just the mean
  precisionDf <- stack(lapply(precisionTool[[sample]], mean))
  colnames(precisionDf) <- c("precision", "tool")

  recallDf <- stack(lapply(recallTool[[sample]], mean))
  colnames(recallDf) <- c("recall", "tool")

  Fscores <- sapply(names(precisionTool[[sample]]), function(tool){
    (2 * precisionTool[[sample]][[tool]] * recallTool[[sample]][[tool]]) /
    (precisionTool[[sample]][[tool]] + recallTool[[sample]][[tool]])
  })
  meanFscores <- colMeans(Fscores)
  FscoreDf <- stack(meanFscores)
  colnames(FscoreDf) <- c("F1score", "tool")

  df <- data.frame(
    tool = rep(names(precisionTool[[sample]]),
               times = sapply(precisionTool[[sample]], length)),
    precision = unlist(precisionTool[[sample]]),
    recall = unlist(recallTool[[sample]])
  )
  df$F1score <- (2 * df$precision * df$recall) / (df$precision + df$recall)

  df$tool <- factor(df$tool, levels=rev(names(colorTools)))
}

```

```

[1] "all"
[1] "old"
[1] "young"

```

In [22]: `# save just means, quartiles and min max`

```

meanF1score_age_df <- NULL

for(age in c("old", "young")){

  precisionDf <- stack(lapply(precisionTool[[age]], mean))
  colnames(precisionDf) <- c("precision", "tool") # precision is mean precision
  precisionDf$precision_min <- stack(lapply(precisionTool[[age]], min))$values
  precisionDf$precision_max <- stack(lapply(precisionTool[[age]], max))$values
  precisionDf$precision_median <- stack(lapply(precisionTool[[age]], median))$values
  precisionDf$precision_sd <- stack(lapply(precisionTool[[age]], sd))$values
  precisionDf$precision_q25 <- stack(lapply(precisionTool[[age]], function(x) quantile(x, probs = 0.25)))$value
  precisionDf$precision_q75 <- stack(lapply(precisionTool[[age]], function(x) quantile(x, probs = 0.75)))$value
}

```



```

recallDf <- stack(lapply(recallTool[[age]], mean))
colnames(recallDf) <- c("recall", "tool")
recallDf$recall_min <- stack(lapply(recallTool[[age]], min))$values
recallDf$recall_max <- stack(lapply(recallTool[[age]], max))$values
recallDf$recall_median <- stack(lapply(recallTool[[age]], median))$values
recallDf$recall_sd <- stack(lapply(recallTool[[age]], sd))$values
recallDf$recall_q25 <- stack(lapply(recallTool[[age]], function(x) quantile(x, probs = 0.25)))$values
recallDf$recall_q75 <- stack(lapply(recallTool[[age]], function(x) quantile(x, probs = 0.75)))$values

Fscores <- sapply(names(precisionTool[[age]]), function(tool){
  (2 * precisionTool[[age]][[tool]] * recallTool[[age]][[tool]]) /
  (precisionTool[[age]][[tool]] + recallTool[[age]][[tool]])
})
meanFscores <- colMeans(Fscores)
FscoreDf <- stack(meanFscores)
colnames(FscoreDf) <- c("F1score", "tool")
FscoreDf$F1score_min <- apply(Fscores, MARGIN = 2, min)
FscoreDf$F1score_max <- apply(Fscores, MARGIN = 2, max)
FscoreDf$F1score_median <- apply(Fscores, MARGIN = 2, median)
FscoreDf$F1score_sd <- apply(Fscores, MARGIN = 2, sd)
FscoreDf$F1score_q25 <- apply(Fscores, MARGIN = 2, function(x) quantile(x, probs = 0.25))
FscoreDf$F1score_q75 <- apply(Fscores, MARGIN = 2, function(x) quantile(x, probs = 0.75))

df <- merge(precisionDf, recallDf, by=c("tool"))
df <- merge(df, FscoreDf, by=c("tool"))

df$tool <- factor(df$tool, levels=rev(names(colorTools)))

# create df for facet plot
meanF1score_age_df <- rbind(meanF1score_age_df, cbind(df, age))
}

```

```

In [24]: meanF1score_age_df_forDepthcomparison <- meanF1score_age_df

meanF1score_age_df_forDepthcomparison <- meanF1score_age_df_forDepthcomparison[!(
  meanF1score_age_df_forDepthcomparison$tool == "Stellarscope" &
  meanF1score_age_df_forDepthcomparison$age == "young"),]

meanF1score_age_df_forDepthcomparison$tool[meanF1score_age_df_forDepthcomparison$tool == "Stellarscope_manualrun"] <- "Stellarscope"
meanF1score_age_df_forDepthcomparison

write.table(meanF1score_age_df_forDepthcomparison, "data/meanF1score_age_df.tsv")

```

	tool	precision	precision_min	precision_max	precision_median	precision_sd	precision_q25	precision_q75
	<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	SoloTE_thr0	0.95192142	0.94222020	0.96006520	0.95183134	0.0028309199	0.95010352	0.95376194
2	SoloTE_thr1	0.97411146	0.96938295	0.97857476	0.97413793	0.0015174692	0.97317248	0.97515288
3	SoloTE_thr2	0.97621203	0.97117202	0.98037383	0.97626511	0.0014354796	0.97529061	0.97733962
4	SoloTE_unique	0.97889982	0.97473785	0.98263448	0.97894737	0.0013391751	0.97799054	0.97999054
5	STARsolo_TE	0.99962085	0.99855072	1.00000000	0.99957699	0.0003158348	0.99955222	1.00000000
6	STARsolo_TE_EM	0.96831395	0.95501874	0.98682477	0.96850881	0.0054239898	0.96473129	0.97229081
7	Stellarscope	0.99528062	0.99202252	0.99768411	0.99536256	0.0009013589	0.99471661	0.99657111
8	SoloTE_thr0	0.08151549	0.07242289	0.08733624	0.08168931	0.0023855791	0.07986814	0.08351066
9	SoloTE_thr1	0.16786837	0.15148579	0.18326693	0.16780293	0.0051702204	0.16482092	0.16982914
10	SoloTE_thr2	0.20608029	0.18163265	0.22599469	0.20616937	0.0068065437	0.20183254	0.21032721
11	SoloTE_unique	0.25825525	0.22649955	0.28229665	0.25839552	0.0094643442	0.25186567	0.26482092
12	STARsolo_TE	0.15531343	0.14042442	0.17017359	0.15529933	0.0047462580	0.15229619	0.15779054
13	STARsolo_TE_EM	0.11403170	0.10774818	0.12142766	0.11394815	0.0021647087	0.11258834	0.11532721

```

In [25]: options(repr.plot.width=9, repr.plot.height=4)

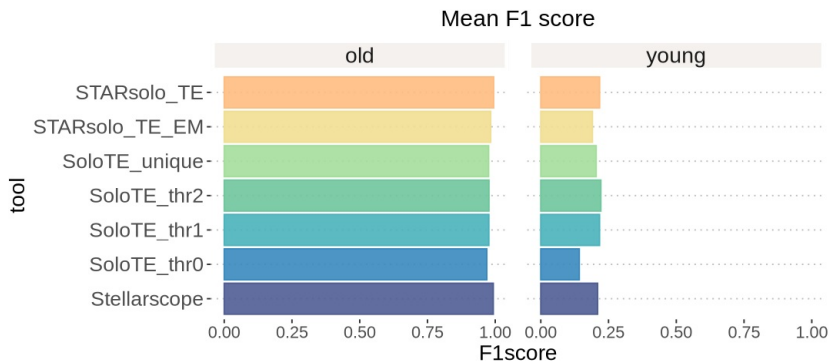
ggplot(meanF1score_age_df, aes(x=F1score, y=tool, color=tool, fill=tool)) +
  facet_wrap(~age, ncol = 2) +
  geom_col(alpha = 0.9) +
  scale_color_manual(values=colorTools) +
  scale_fill_manual(values=colorTools) +

```

```

theme_pubclean() + ggtitle(paste0("Mean F1 score")) +
guides(fill="none", color="none") +
xlim(c(0,1)) +
theme(text=element_text(size=17),
      plot.title = element_text(size=18, hjust=0.5),
      plot.subtitle = element_text(size=18, hjust=0.5),
      axis.text.y = element_text(size=16),
      axis.text.x = element_text(size=13),
      strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
      strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
      panel.spacing = unit(1, "lines"))
ggsave(paste0("figures_", dataset_id, "/meanF1_tool_byAge_horiz.png"), height=3.2, width=7)
ggsave(paste0("figures_", dataset_id, "/meanF1_tool_byAge_horiz.pdf"), device="pdf", height=3.2, width=7)

```

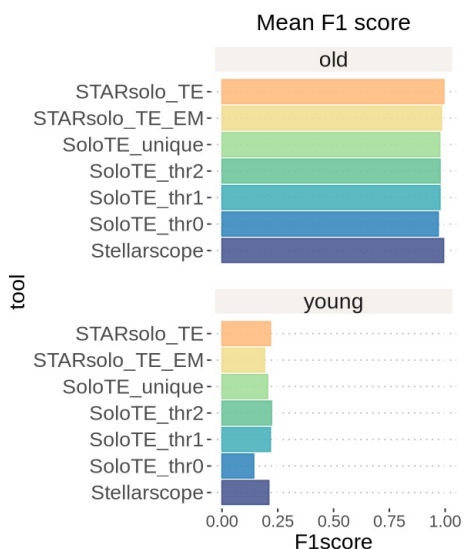


```

In [27]: options(repr.plot.width=5, repr.plot.height=6)

ggplot(meanF1score_age_df, aes(x=F1score, y=tool, color=tool, fill=tool)) +
  facet_wrap(~age, ncol = 1) +
  geom_col(alpha = 0.9) +
  scale_color_manual(values=colorTools) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Mean F1 score")) +
  guides(fill="none", color="none") +
  xlim(c(0,1)) +
  theme(text=element_text(size=17),
        plot.title = element_text(size=18, hjust=0.5),
        plot.subtitle = element_text(size=18, hjust=0.5),
        axis.text.y = element_text(size=16),
        axis.text.x = element_text(size=13),
        strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
        strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
        panel.spacing = unit(1, "lines"))
ggsave(paste0("figures_", dataset_id, "/meanF1_tool_byAge_vert.pdf"), device="pdf", height=6, width=5)

```



```

In [29]: options(repr.plot.width=8, repr.plot.height=6)

precision_plot <- ggplot(meanF1score_age_df, aes(x=precision, y=tool, color=tool, fill=tool)) +
  facet_wrap(~age, ncol = 1) +
  geom_col(alpha = 0.9) +
  scale_color_manual(values=colorTools) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Mean precision")) +
  guides(fill="none", color="none") +
  xlim(c(0,1)) +
  ylab("") +
  theme(text=element_text(size=17),

```

```

    plot.title = element_text(size=18, hjust=0.5),
    plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_text(size=16),
    axis.text.x = element_text(size=13),
    strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    panel.spacing = unit(1, "lines"))

recall_plot <- ggplot(meanF1score_age_df, aes(x=recall, y=tool, color=tool, fill=tool)) +
  facet_wrap(~age, ncol = 1) +
  geom_col(alpha = 0.9) +
  scale_color_manual(values=colorTools) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Mean recall")) +
  guides(fill="none", color="none") +
  xlim(c(0,1)) +
  ylab("") +
  theme(text=element_text(size=17),
    plot.title = element_text(size=18, hjust=0.5),
    plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_blank(),
    axis.text.x = element_text(size=13),
    strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    panel.spacing = unit(1, "lines"))

f1_plot <- ggplot(meanF1score_age_df, aes(x=F1score, y=tool, color=tool, fill=tool)) +
  facet_wrap(~age, ncol = 1) +
  geom_col(alpha = 0.9) +
  scale_color_manual(values=colorTools) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Mean F1score")) +
  guides(fill="none", color="none") +
  xlim(c(0,1)) +
  ylab("") +
  theme(text=element_text(size=17),
    plot.title = element_text(size=18, hjust=0.5),
    plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_blank(),
    axis.text.x = element_text(size=13),
    strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    panel.spacing = unit(1, "lines"))

plot <- precision_plot + recall_plot
plot

ggsave(paste0("figures_simulated_mm_RA/meanPrecisionRecall_tool_byAge.pdf"),
  device="pdf", height=6, width=8)

```

\$all

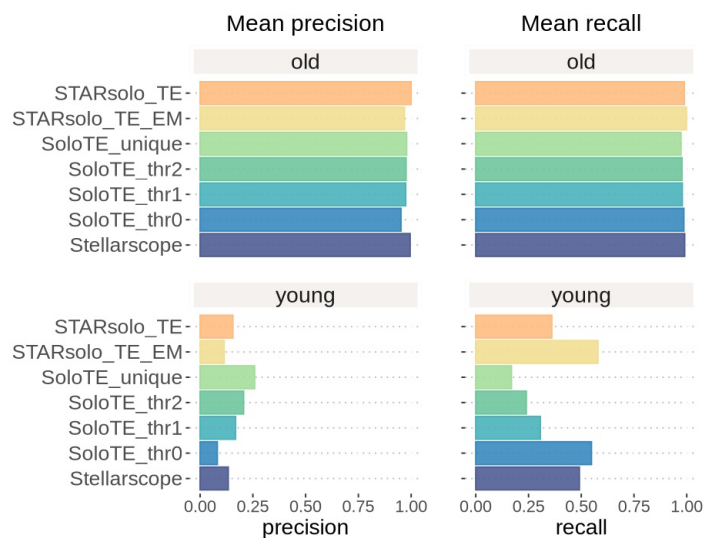
An object of class Seurat
 4947 features across 500 samples within 1 assay
 Active assay: RNA (4947 features, 0 variable features)
 2 layers present: counts, data

\$old

An object of class Seurat
 3030 features across 500 samples within 1 assay
 Active assay: RNA (3030 features, 0 variable features)
 2 layers present: counts, data

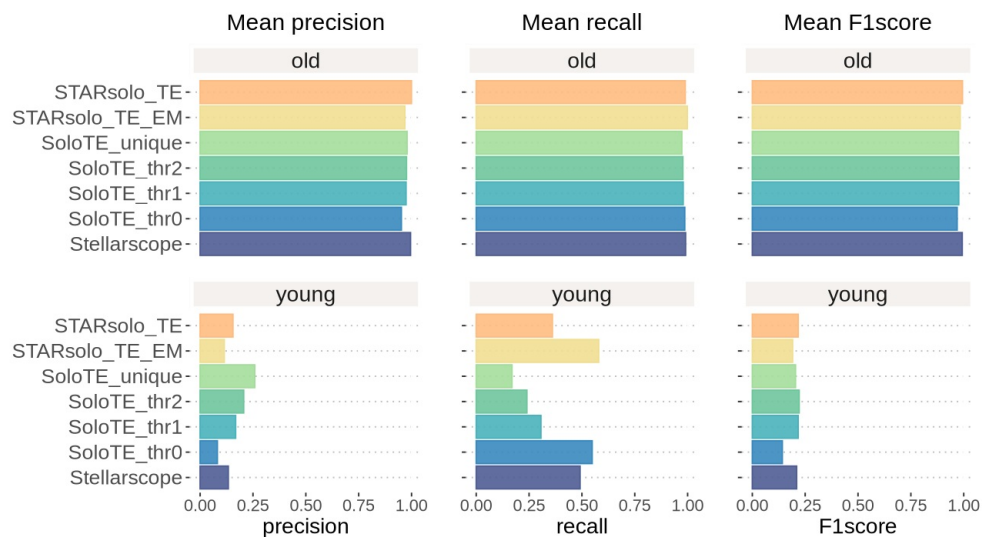
\$young

An object of class Seurat
 2030 features across 500 samples within 1 assay
 Active assay: RNA (2030 features, 0 variable features)
 2 layers present: counts, data



```
In [30]: options(repr.plot.width=11, repr.plot.height=6)

plot + f1_plot
ggsave(paste0("figures_simulated_mm_RA/meanPrecisionRecallF1score_tool_byAge.pdf"),
       device="pdf", height=6, width=11)
```



```
In [30]: saveRDS(objList, paste0("data/objList_", dataset_id, ".RDS"))
```

Precision recall curve

```
In [84]: pad_sparse_matrix <- function(mat, all_genes) {
  missing_genes <- setdiff(all_genes, rownames(mat))
  if (length(missing_genes) > 0) {
    zero_mat <- Matrix(0, nrow = length(missing_genes), ncol = ncol(mat), sparse = TRUE,
                      dimnames = list(missing_genes, colnames(mat)))
    mat <- rbind(mat, zero_mat)
  }
  # Reorder rows to match all_genes
  mat <- mat[all_genes, , drop = FALSE]
```

```

    return(mat)
}

```

```

In [44]: precisionRecallDfTool <- list()
prCurves <- list()
rocCurves <- list()
auprc_df <- list()

for(sample in samples){

  auprc_df[[sample]] <- data.frame()

  precisionRecallDfTool[[sample]] <- NULL

  gt_matrix <- GetAssayData(splatter_objs[[sample]], layer='count')

  prCurves[[sample]] <- list()
  rocCurves[[sample]] <- list()

  for(tool in names(objList)){

    inferred_matrix <- GetAssayData(objList[[tool]][[sample]], layer='count')
    gc()

    # add missing loci to both matrices
    all_genes <- union(rownames(gt_matrix), rownames(inferred_matrix))
    gt_padded <- pad_sparse_matrix(gt_matrix, all_genes)
    inferred_padded <- pad_sparse_matrix(inferred_matrix, all_genes)

    # Flatten to vectors
    gt_vec <- as.vector(gt_padded)
    inferred_vec <- as.vector(inferred_padded)

    # Binarize ground truth: 1 if expressed, 0 otherwise
    gt_binary <- as.numeric(gt_vec > 0)

    # Positive class scores: inferred values for ground truth = 1
    # Negative class scores: inferred values for ground truth = 0
    pos_scores <- inferred_vec[gt_binary == 1]
    neg_scores <- inferred_vec[gt_binary == 0]

    # Create PR curve
    pr_obj <- pr.curve(
      scores.class0 = inferred_vec,
      weights.class0 = gt_binary,
      curve = TRUE,
      max.compute = TRUE,
      min.compute = TRUE,
      rand.compute = TRUE
    )
    prCurves[[sample]][[tool]] <- pr_obj

    rocCurves[[sample]][[tool]] <- roc.curve(scores.class0 = inferred_vec, weights.class0 = gt_bina
      max.compute = TRUE, min.compute = TRUE, rand.compute = TRUE)

    auprc <- pr_obj$auc.integral

    auprc_df[[sample]] <- rbind(
      auprc_df[[sample]],
      data.frame(
        sample = sample,
        tool = tool,
        AUPRC = auprc
      )
    )

    ## Manually plot curves
    thresholds <- c(1e-10, seq(1, 9, length.out = 9), seq(10, 100, length.out = 10))
    precision <- numeric(length(thresholds))
    recall <- numeric(length(thresholds))
    F1score <- numeric(length(thresholds))
    # Create binary labels: 1 = expressed, 0 = not expressed
    # Based on ground truth
    labels <- ifelse(gt_binary > 0, 1, 0)

    for (i in seq_along(thresholds)) {
      t <- thresholds[i]
      pred_binary <- ifelse(inferred_vec >= t, 1, 0)

      TP <- sum(pred_binary == 1 & labels == 1)
      FP <- sum(pred_binary == 1 & labels == 0)
      FN <- sum(pred_binary == 0 & labels == 1)
    }
  }
}

```

```

precision[i] <- ifelse(TP + FP == 0, 1, TP / (TP + FP))
recall[i] <- ifelse(TP + FN == 0, 0, TP / (TP + FN))
F1score[i] <- ( 2 * precision[i] * recall[i] ) / ( precision[i] + recall[i] )
}

df_plot <- as.data.frame(cbind(precision, recall, F1score, thresholds))
mini_df <- df_plot[1:5,]

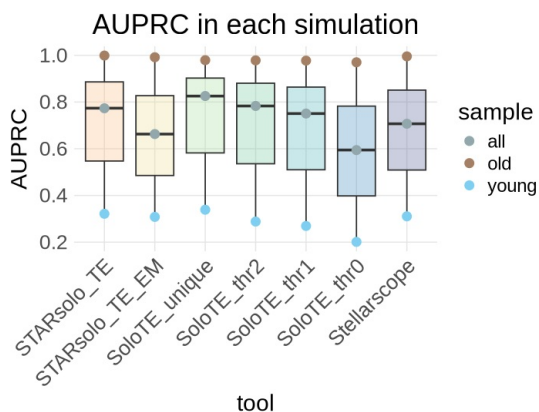
df_plot <- cbind(df_plot, tool)
precisionRecallDfTool[[sample]] <- rbind(precisionRecallDfTool[[sample]], df_plot)
}

```

```

In [45]: options(repr.plot.width=6, repr.plot.height=4.5)
df <- bind_rows(auprc_df)
df$tool <- factor(df$tool, levels = names(colorTools))
ggplot(df, aes(tool, AUPRC)) +
  geom_boxplot(aes(fill=tool), alpha = 0.3) +
  geom_jitter(width = 0, size=3, alpha = 1, aes(color=sample)) +
  scale_color_manual(values = colorSamples) +
  scale_fill_manual(values = colorTools) +
  ggtitle("AUPRC in each simulation") +
  guides(fill = "none") +
  theme_minimal() +
  theme(text=element_text(size=18),
        plot.title = element_text(hjust=0.5),
        plot.subtitle = element_text(hjust=0.5),
        axis.text.y = element_text(size=16),
        axis.text.x = element_text(size=16, angle=45, hjust=1),
        panel.grid.major = element_line(colour = "grey90", linewidth = 0.4),
        panel.grid.minor = element_line(colour = "white", linewidth = 0.2),
        legend.position="right")
ggsave(paste0("figures_", dataset_id, "/AUPRC_by_toolAndSim.pdf"), width = 6, height= 4.5)

```



```

In [71]: for(sample in samples){
  precisionRecallDfTool[[sample]]$thresholds <- as.numeric(gsub(precisionRecallDfTool[[sample]]$thresholds, p

  precisionRecallDfTool[[sample]] <- precisionRecallDfTool[[sample]][(precisionRecallDfTool[[sample]]$thresholds
    (precisionRecallDfTool[[sample]]$tool == "STARsolo_TE_EM"), ]
}

```

```

In [72]: options(repr.plot.width=9, repr.plot.height=7)

for(sample in samples){

  mini_df <- precisionRecallDfTool[[sample]][(precisionRecallDfTool[[sample]]$thresholds %in% c(0, 1,5,10

  # Plot manual PR curve
  show( ggplot(precisionRecallDfTool[[sample]], aes(x=recall, y=precision, color=tool)) +
    geom_line(size=1, alpha=0.8) +
    geom_point(size=0.7) +
    scale_color_manual(values=colorTools) +
    ggrepel::geom_text_repel(data = mini_df, aes(label = thresholds), size= 5) +
    xlim(c(0,1)) +
    ylim(c(0,1)) +
    theme_pubr() +
    theme(text=element_text(size=20),
          plot.title = element_text(hjust=0.5),
          plot.subtitle = element_text(hjust=0.5),
          axis.text.y = element_text(size=15),
          axis.text.x = element_text(size=15),
          panel.grid.major = element_line(colour = "grey90", linewidth = 0.3),
          panel.grid.minor = element_line(colour = "grey90", linewidth = 0.2),

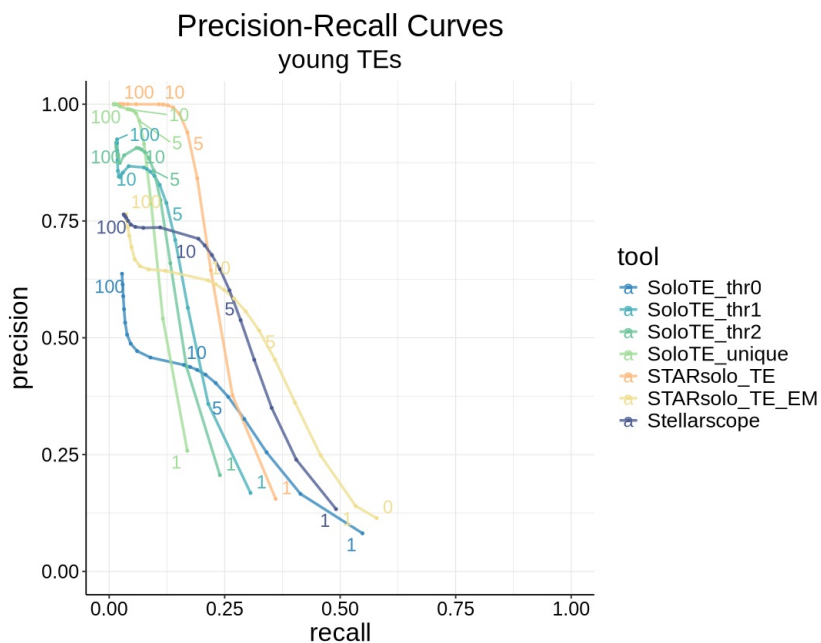
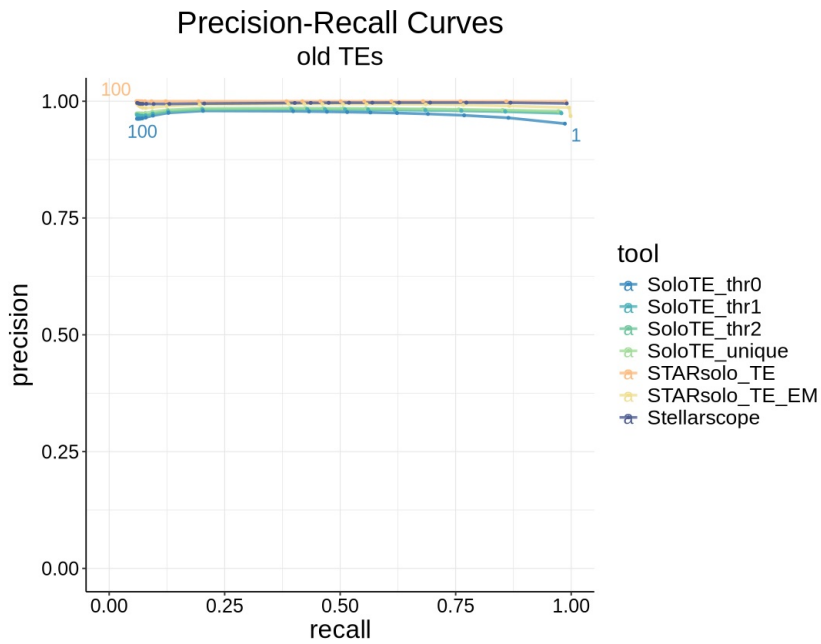
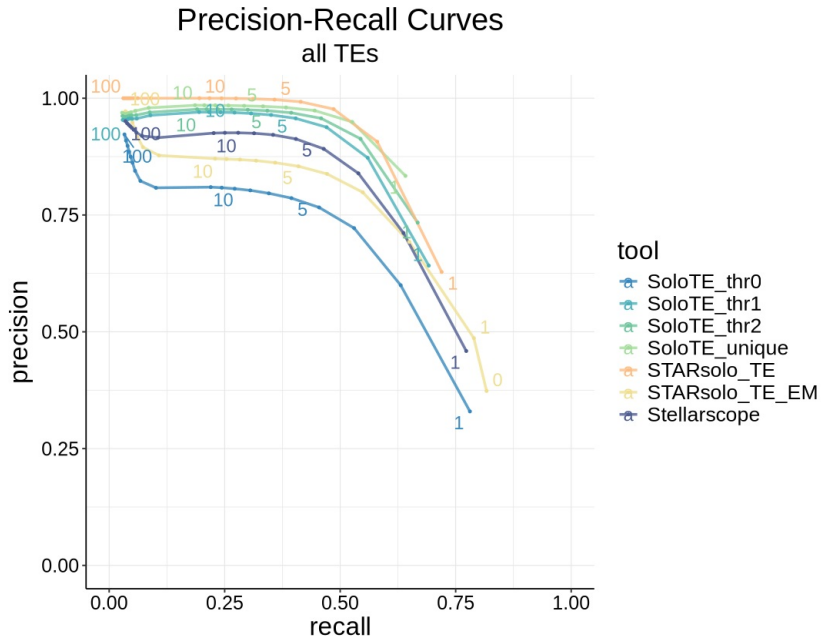
```

```

        legend.position="right") +
        ggtitle("Precision-Recall Curves", subtitle=paste0(sample," TEs")) )

ggsave(paste0("figures_simulated_mm_RA_",sample,"/PrecisionRecallCurve_byTool.png"), height=7, width=9)
ggsave(paste0("figures_simulated_mm_RA_",sample,"/PrecisionRecallCurve_byTool.pdf"), device="pdf", heig
}

```



```

In [76]: options(repr.plot.width=10, repr.plot.height=7)

for(sample in samples){

  # best threshold for each tool
  best_pts <- precisionRecallDfTool[[sample]] %>%
  group_by(tool) %>%
  slice_max(Flscore, n = 1, with_ties = FALSE) %>%
  ungroup()

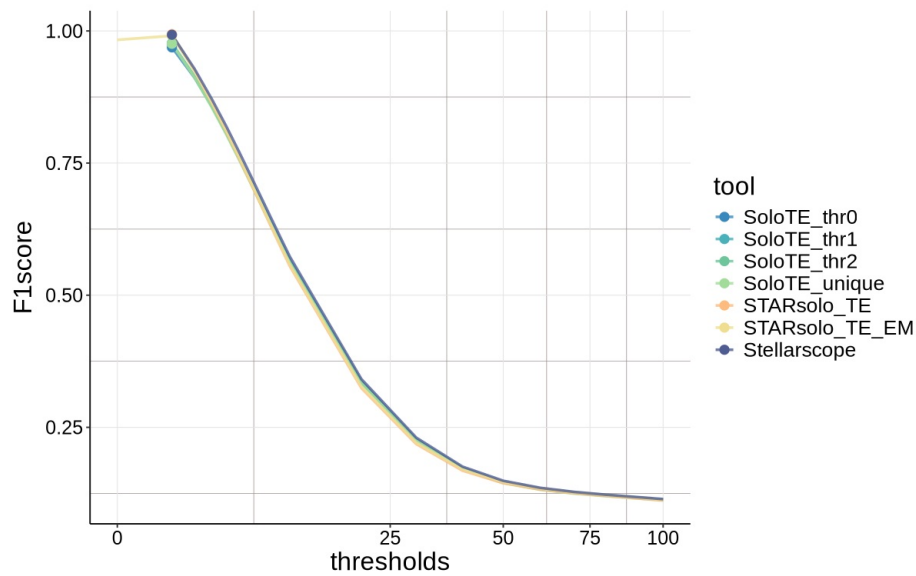
  show(ggplot(precisionRecallDfTool[[sample]], aes(thresholds, Flscore, color = tool)) +
    geom_line(size=1, alpha=0.8) +
    #geom_point() +
    scale_color_manual(values=colorTools) +
    #ggrepel::geom_text_repel(data = mini_df, aes(label = thresholds), size= 5) +
    geom_point(data=best_pts, size=3) +
    ggrepel::geom_text_repel(
      data = best_pts,
      aes(label = paste0("t=", thresholds)),
      show.legend = FALSE,
      size = 5
    ) +
    ylim(c(0,1)) +
    scale_x_continuous(transform = "sqrt") +
    theme_pubr() +
    theme(text=element_text(size=20),
      plot.title = element_text(hjust=0.5),
      plot.subtitle = element_text(hjust=0.5),
      axis.text.y = element_text(size=15),
      axis.text.x = element_text(size=15),
      panel.grid.major = element_line(colour = "grey90", linewidth = 0.3),
      panel.grid.minor = element_line(colour = "#968787", linewidth = 0.2),
      legend.position="right") +
    ggtitle("F1 scores", subtitle=paste0(sample," TEs")) )

  ggsave(paste0("figures_simulated_mm_RA_",sample,"/F1scores_byThreshold_byTool.png"), height=7, width=9)
  ggsave(paste0("figures_simulated_mm_RA_",sample,"/F1scores_byThreshold_byTool.pdf"), device="pdf", height=7, width=9)

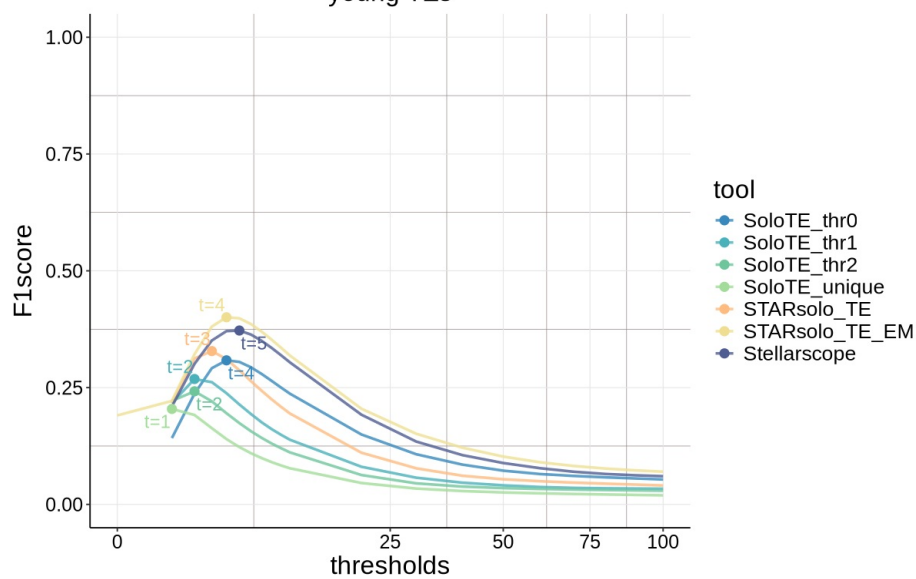
  show(ggplot(precisionRecallDfTool[[sample]], aes(thresholds, Flscore, color = tool)) +
    geom_line(size=1, alpha=0.8) +
    scale_color_manual(values=colorTools) +
    #ggrepel::geom_text_repel(data = mini_df, aes(label = thresholds), size= 5) +
    geom_point(data=best_pts, size=3) +
    ggrepel::geom_text_repel(
      data = best_pts,
      aes(label = paste0("t=", thresholds)),
      show.legend = FALSE,
      size = 5
    ) +
    #ylim(c(0,1)) +
    #scale_x_log10() +
    scale_x_continuous(transform = "sqrt") +
    theme_pubr() +
    theme(text=element_text(size=20),
      plot.title = element_text(hjust=0.5),
      plot.subtitle = element_text(hjust=0.5),
      axis.text.y = element_text(size=15),
      axis.text.x = element_text(size=15),
      panel.grid.major = element_line(colour = "grey90", linewidth = 0.3),
      panel.grid.minor = element_line(colour = "#968787", linewidth = 0.2),
      legend.position="right") +
    ggtitle("F1 scores", subtitle=paste0(sample," TEs")) )
  ggsave(paste0("figures_simulated_mm_RA_",sample,"/F1scores_byThreshold_byTool_noConstraints.pdf"), device="pdf", height=7, width=9)
}

```

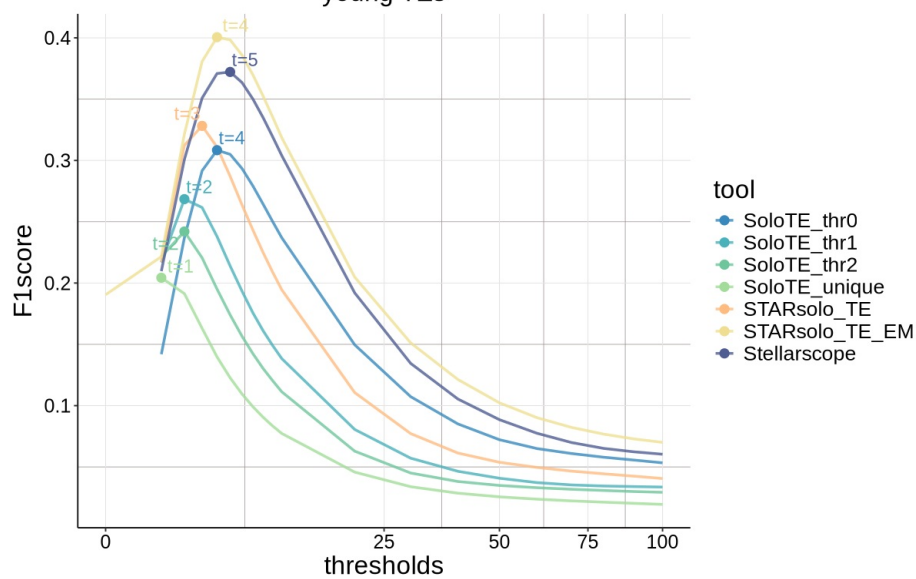

F1 scores
old TEs



F1 scores
young TEs



F1 scores
young TEs



```
In [77]: options(repr.plot.width=9, repr.plot.height=7)

for(sample in samples){

  mini_df <- precisionRecallDfTool[[sample]][(precisionRecallDfTool[[sample]]$thresholds %in% c(0,1,5,10,

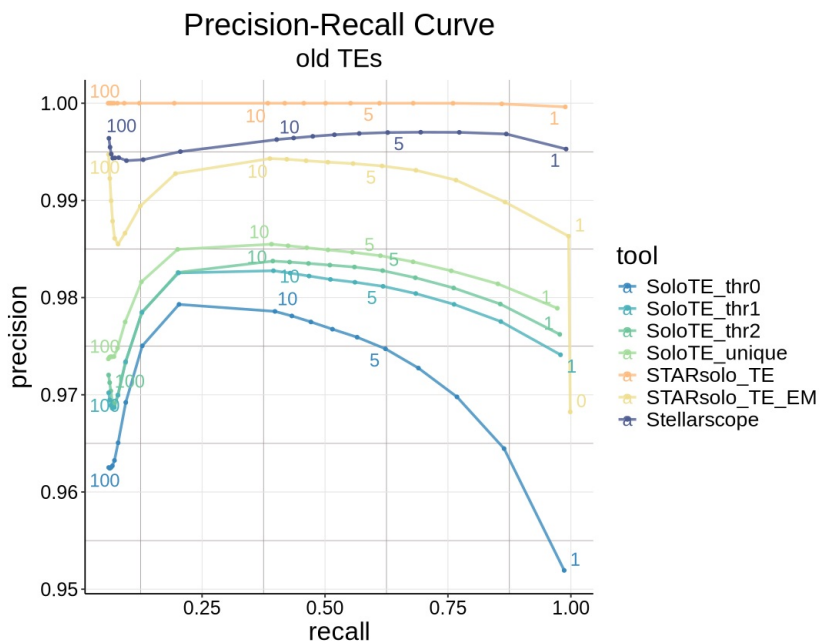
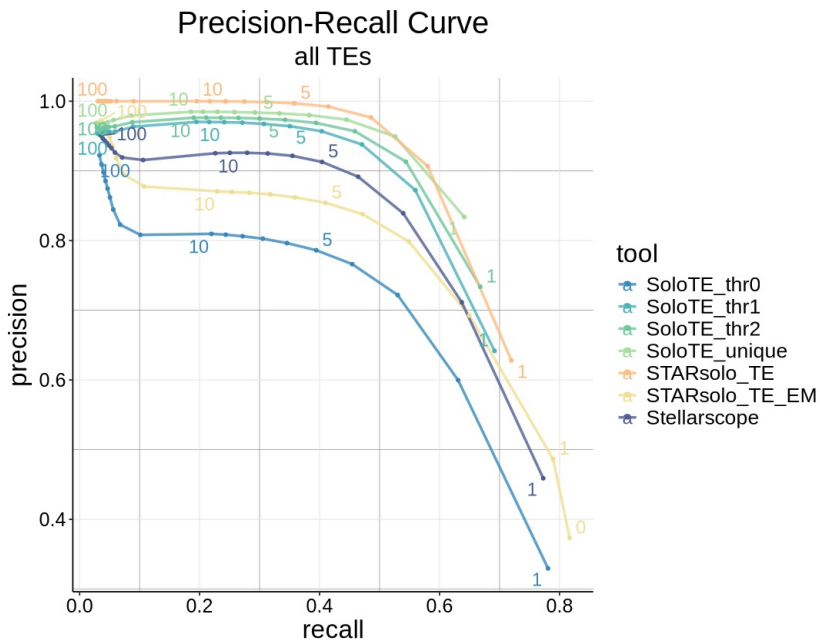
  # Plot manual PR curve
  show( ggplot(precisionRecallDfTool[[sample]], aes(x=recall, y=precision, color=tool)) +
```

```

geom_line(size=1, alpha=0.8) +
geom_point(size=1) +
scale_color_manual(values=colorTools) +
ggrepel::geom_text_repel(data = mini_df, aes(label = thresholds), size=5) +
# xlim(c(0,1)) +
# ylim(c(0,1)) +
theme_pubr() +
  theme(text=element_text(size=20),
        plot.title = element_text(hjust=0.5),
        plot.subtitle = element_text(hjust=0.5),
        axis.text.y = element_text(size=15),
        axis.text.x = element_text(size=15),
        panel.grid.major = element_line(colour = "grey90", linewidth = 0.3),
        panel.grid.minor = element_line(colour = "#968787", linewidth = 0.2),
        legend.position="right") +
ggtitle("Precision-Recall Curve", subtitle=paste0(sample," TEs"))

ggsave(paste0("figures_simulated_mm_RA_",sample,"/PrecisionRecallCurve_byTool_noaxis.png"), height=7, w.
ggsave(paste0("figures_simulated_mm_RA_",sample,"/PrecisionRecallCurve_byTool_noaxis.pdf"), device="pdf"
}

```



Precision-Recall Curve
young TEs

